PROJECT-BASED LEARNING

PHASE-II EVALUATION

InternMatch

Real-Time Internship Matching System

Team ID: DSCPP-III-2026-T024

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team and Mentor Details

Team Information

Team ID: DSCPP-III-2026-T024
Semester: 3rd
Section: ML1 / ML2 / Cyber
Domain: DSA and OOP in C++

Team Members

Jigyasa Rana – 2510380058 (Lead)
Masoom Singh – 2510370548
Kashvi Pandey – 2510012207

Mentor

Dr. Siddhant Thapliyal

Interactions completed: 9

Brief discussion: Introduce the team, project domain, mentor and responsibilities.

Phase-I Recap and Feedback Incorporated

Phase-I Foundation

We made the `Student` and `Internship` classes in C++ and got real job data from the Arbeitnow API using `libcurl` and `nlohmann/json`.

Feedback received

"The idea is good; make the project more useful and developed."

Improvements in Phase-II

- Our own linked list to store internships.
- `Student` inherits from `Person`.
- Better skill matching (case does not matter).
- Filter: 28 of 300 jobs kept.
- Final score = 80% skill match + 20% role fit.
- `Unverified` if CGPA/branch is missing.

Note: Role Fit is only a simple guess from the job title.

Problem Statement and Objectives

Problem statement

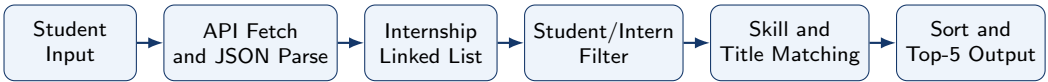
Checking many job listings one by one to find the ones that match your skills takes a lot of time. InternMatch gets live listings, keeps the student/internship ones, and shows the best 5 matches based on your skills.

Phase-II objectives

1. Get live job listings from an API and keep only student/internship jobs.
2. Use C++ classes and inheritance for person, student and internship.
3. Store all internships in our own linked list.
4. Match the student's skills with the job title/tags, give a score, and show the top 5.
5. Show `Unverified` when the job data has no CGPA or branch requirement.

Limitation: the API does not give CGPA/branch, so we show `Unverified` instead of guessing.

System Architecture and Workflow



Core modules

- `Person` and `Student`
- `Internship`
- `LinkedList`
- `MatchEngine`

Output indicators

- Skill coverage percentage
- Title-based role-fit estimate
- Combined estimated match score (80% skills + 20% role fit)
- Eligibility: `Unverified` when data is missing

The linked list stores all fetched internships. Matching and ranking are done on them.

Module Design and Functional Requirements

Major modules

1. `Person` – stores the name.
2. `Student` – stores branch, year, CGPA, skills; inherits from `Person`.
3. `Internship` – stores company, role, tags and requirements.
4. `LinkedList` – adds internships and reads them one by one.
5. `MatchEngine` – cleans text, checks skills and calculates match %.

The API has no CGPA/branch data, so eligibility is shown as Unverified.

Functional requirements

- Take student details as input.
- Get and read live API data.
- Store listings in a linked list.
- Keep only student/internship jobs.
- Calculate skill match and role fit.
- Sort and display up to five results.
- Do not guess eligibility when data is missing.

Technology Stack and Methodology

Technology stack

- Language: C++17
- Data source: Arbeitnow job-board API
- Getting data from internet: `libcurl`
- Reading JSON data: `nlohmann/json`
- DSA/OOP: linked list, classes, inheritance, searching and sorting
- Tools: VS Code, Terminal, Git and GitHub

All code is in C++; libcurl and nlohmann/json are C++ libraries.

Development workflow

1. Define requirements and modules.
2. Implement classes and linked list.
3. Fetch and parse API data.
4. Match skills and rank results.
5. Compile and test with sample student profiles.
6. Track stable changes using Git/GitHub.

Implementation of DSA and OOP Concepts

Concept	What the project does	Source module
Singly linked list	Each node stores one `Internship` and links to the next node. New ones are added at the end and we read them one by one.	`LinkedList.h/.cpp`
Inheritance	`Student` gets the `name` variable from `Person`, so we do not write it again.	`Person.h`, `Student.h`
Encapsulation	Data is kept private inside classes and used through functions.	`Student`, `Internship`
String matching	Text is made lowercase and skills are matched as whole words, so "c" does not match inside "c++".	`MatchEngine.cpp`
Ranking	Final score = 80% skill match + 20% role fit. Results are sorted high to low using C++ sort.	`main.cpp` / matching module

No Machine Learning or GUI in Phase-II. Ranking uses C++ sort and a simple score formula.

Development and Implementation Progress

Module	Status	Work completed / limitation
`Student/Person`	Implemented	`Student` inherits from `Person`; details are taken as console input.
`Internship`	Implemented	Stores company, role, tags and requirements.
Live API and JSON	Implemented	Gets live listings and reads company, title and tags. The count changes on each run.
Student/internship filter	Implemented	Keeps only student/internship jobs (sample run: 28 of 300).
`LinkedList`	Implemented	Our own linked list: add and read internships.
Skill coverage	Implemented	Finds what % of the student's skills appear in the job title/tags.
Ranking	Implemented with simple rule	Final score = 80% skill match + 20% role fit; shows top 5.
CGPA/branch verification	Data-limited	API does not give CGPA/branch, so status is `Unverified`.
Menu-driven interface	Planned (Phase-III)	Right now it runs in the console.

All main Phase-II features work. Real eligibility check is not possible because the data is not available.

Working Prototype: Observed Output

Implemented in the prototype

- Student profile input.
- Live data from API.
- Internships stored in a linked list.
- Filter: 28 of 300 jobs kept.
- Skill match, role fit and final score.
- Top 5 results with eligibility status.

Scores are only estimates, not a guarantee of selection.



Testing, Results and Validation

Test	Expected result	Observed evidence	Status
Compilation	Project compiles without errors.	Code compiled and the program ran.	Pass
Live API/JSON	Get and read live job listings.	Fetched 300 listings; 28 stayed after the filter. Numbers change each run.	Pass
Skill coverage	Score shows how many entered skills matched.	3 of 4 skills (python, sql, git) matched = 75%.	Pass
Ranking	Higher score comes first.	Top 5 shown high to low: 74, 74, 60, 40, 34.	Pass
Missing eligibility data	Do not guess eligibility.	Shown as `Unverified` when CGPA/branch was missing.	Pass
Input robustness	Wrong input should not crash the program.	Not fully tested yet.	Pending

Limitations: no eligibility data in API, role fit is only an estimate, and live listings change.

GitHub Repository and Development Activity

Repository

- `https://github.com/masoomsingh022/InternMatch`
- Final branch: `main`
- Code, README and commits are on GitHub.
- Backup files are ignored using `.gitignore`.

We submit the `main` branch. The `integration-test` branch was only for testing.

Version-control practice

- Compile and run after every change.
- Check the code changes before committing.
- Use meaningful commit messages.
- Check for conflicts before merging branches.

Individual Contributions

Member	Role	Technical contribution
Jigyasa Rana	Team Lead	Designed `Student.h` and `Person.h` in Phase-I; built the `Person-Student` inheritance in Phase-II.
Masoom Singh	Member	Handled JSON/API data and the linked list in Phase-I; wrote the skill-matching logic in `MatchEngine` in Phase-II.
Kashvi Pandey	Member	Built `Internship.h` in Phase-I; working on input validation and multi-profile testing in Phase-II.

Individual evaluation preparation

Each member should be ready to explain their own files and functions. Check the real GitHub commit count live instead of guessing it here.

Challenges, Solutions and Phase-III Roadmap

Challenges

- API has no CGPA/branch requirements.
- Simple search gives wrong matches ("c" in "c++").
- Live listings change between runs.
- Filter can let in non-tech student jobs.

Solutions in Phase-II

- Show `Unverified` when requirements are missing.
- Lowercase text, whole-word matching.
- Tested with real API data.

Phase-III roadmap

1. Join all parts and review the code.
2. Make a menu-driven interface.
3. Check user input properly.
4. Improve the filter so non-tech jobs rank lower.
5. Test more profiles and edge cases.
6. Finish documentation and demo.

Expected Outcomes and References

Expected outcomes

- A C++ app that gets live internships and ranks them by skill match and role fit.
- Clean OOP design and our own linked list.
- Showing `Unverified` honestly when data is missing.
- Later: menu-driven interface and better eligibility data.

References

1. Arbeitnow Job Board API: `https://www.arbeitnow.com/api/job-board-api`
2. libcurl documentation: `https://curl.se/libcurl/`
3. nlohmann/json: `https://github.com/nlohmann/json`
4. B. Stroustrup, *The C++ Programming Language*.

Thank You
Questions and Suggestions